

CSCE 411

Divide and Conquer Practice

1. Give an algorithm which takes as input an array $A[1, \dots, n]$ of n *distinct* integers (some of which may be negative) sorted in increasing order and determines whether or not there exists an integer $1 \leq i \leq n$ such that $A[i] = i$. Your algorithm should run in $\Theta(\log n)$ time.
2. Suppose that you are given a sorted array A of n distinct integers and an integer x . You know that x is in A and you want to find its index. That is, you know that there exists some $t \in \mathbb{Z}^+$ such that $A[t] = x$, and you want to find t . Give an $O(\log t)$ time algorithm to find the index t such that $A[t] = x$. (Note that we can determine this in $O(\log n)$ time using binary search, but this may not be fast enough, because we are not guaranteed that $\log n \in O(\log t)$).
3. Suppose that you are given as input an n -digit number in base-10 (represented as a string) and want to compute the same number in base-3 (in base-3 the only valid digits are 0, 1, and 2, and a string of digits $x_1x_2 \dots x_n$ represents the number $\sum_{i=0}^{n-1} 3^i x_{n-i}$).
 - (a) Give a $\Theta(n^2)$ time algorithm for this problem. You do not have to prove the correctness of your algorithm, but you should carefully analyze the runtime.
 - (b) Now give an $o(n^2)$ time algorithm (Hint: Divide and conquer)
 - (c) Now give an $O(n \log^2 n)$ time algorithm for the problem (Hint: Use repeated squaring to avoid a recursive call)
4. We say that an array $A[1, \dots, n]$ of integers is a **valley** if there exists some index i such that for every $1 \leq j < i$ we have $A[j] > A[j+1]$, and for every $i \leq j < n$ we have $A[j] < A[j+1]$. That is, a valley consists of a strictly decreasing sequence followed by a strictly increasing sequence. For example,

$$[7, 5, 4, -2, -7, -15, 1, 3, 10]$$

is a valley, but

$$[5, 6, 8, 4, 2]$$

is not. Give and analyze an algorithm which takes as input an array A which is a valley, as well as an integer x and returns either

- An index j such that $A[j] = x$, or
- -1 if x is not in A .

You may assume that all of the elements of A are distinct. Your algorithm must run in $O(\log n)$ time.